CS177 Bioinformatics: Software Development and Applications

Lecture 4: Markov Chains and Hidden Markov Models

Jie Zheng (郑杰)

PhD, Associate Professor

School of Information Science and Technology (SIST), ShanghaiTech University

Spring, 2025

Learning objectives

- At the end of this lecture, you will be able to:
  - Understand and explain some basic concepts and notations of Markov chains
  - Use Markov chains to model DNA sequences
  - Explain concepts and algorithms of hidden Markov models (HMMs)
  - Explain the expectation maximization (EM) algorithm
  - List a few applications of HMMs in bioinformatics

Markov Chains

Definition of Markov chain

- Consider a discrete stochastic process, when monitored over time, yielding a sequence, $x_0, x_1, x_2, \ldots$ of **states** (or **values**) where x_t is the observed state (or value) at time instant *t*
- Let the **probabilities of states** at the time instant *t* be denoted by $\boldsymbol{\pi}_t$. The corresponding sequence of state probabilities, $(\pi_0, \pi_1, \pi_2, \pi_3, \ldots)$, is referred to as a **Markov chain**
- If the sample space of states, Ω, is indexed, then the state vector is denoted as:

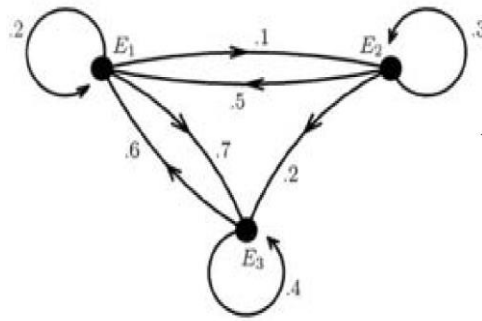
$$\pi_t = \left(\pi_t^{(k)} : k \in \Omega\right)$$

State transition

- The Markov chain is determined by the transitions of one state to another state at the next instant
- Let $a_{k,l}^{(t)}$ be the **transition probability** from state k to state l at instant t
- The **state-transition matrix** at instant t is given by

$$\mathbf{A}^t = \left\{a_{k,l}^{(t)}\right\}_{|\Omega|\times|\Omega|}, \quad \text{for } t = 0,1,2,\ldots$$

- If $\mathbf{A}^t$ is independent of time, the Markov chain is said to be **stationary**. A stationary Markov chain is represented by a single transition matrix $\mathbf{A}$
- **State-transition diagram** indicates the states in the stochastic system and the state transition probabilities



$$\mathbf{A} = \begin{bmatrix} 0.20 & 0.10 & 0.70 \\ 0.50 & 0.30 & 0.20 \\ 0.60 & 0.00 & 0.40 \end{bmatrix}$$

First-order Markov chains

- In a **first-order Markov chain**, state transitions depend only on the previous state, thus the states are entirely determined by the initial state probability and the state transition matrix. The probability of the sequence is then given by

$$
\begin{aligned}
f(x_n, x_{n-1}, \ldots, x_0) &= f(x_n | x_{n-1}, \ldots, x_0) f(x_{n-1} | x_{n-2}, \ldots, x_0) \ldots f(x_1 | x_0) f(x_0) \\
&= f(x_n | x_{n-1}) f(x_{n-1} | x_{n-2}) \ldots f(x_1 | x_0) f(x_0) \\
&= a_{x_{n-1}, x_n} a_{x_{n-2}, x_{n-1}} \ldots a_{x_0, x_1} \pi_0 \\
&= \pi_0 \prod_{t=1}^{n} a_{x_{t-1}, x_t}
\end{aligned}
$$

- A (first-order, stationary) Markov chain $M = (\Omega, \pi_0, \mathbf{A})$ is a stochastic process defined by a set of states, initial probabilities π_0 of states, and state-transition probabilities $\mathbf{A} = \{a_{k,l}\}_{k,l \in \Omega}$

Markov models of DNA sequences (1)

- Given a sequence $x = (x_1, x_2, \cdots, x_n) \in \Omega^n$, each element is presumed to correspond to a random variable $X \in \Omega$ of a stochastic system
- Suppose the first-order Markov property is obeyed, then

$$P(x) = f(x_n | x_{n-1}) f(x_{n-1} | x_{n-2}) \ldots f(x_2 | x_1) f(x_1)$$

$$= P(x_1) \prod_{i=2}^{n} a_{x_{i-1} x_i}$$

- The model is assumed to be stationary, i.e. state-transition probabilities do not depend on the location in the sequence
- If $a_{k,l}$ are transition probabilities from state k to l, the likelihood of the sequence is:

$$P(x) = P(x_1) \prod_{k,l \in \Omega} a_{k,l}^{n_{k,l}}$$

where $n_{k,l}$ is the number of transitions from state k to l that occurred in the sequence

Markov models of DNA sequences (2)

- A Markov chain for DNA sequence can be drawn as in Fig. A, where there is a state for each of the 4 DNA bases: A, C, G and T
- In addition, **begin** and **end** states can be added to the Markov chain for modeling both ends of a sequence (as in Fig. B).
- Then, the probability of the first letter in the sequence being k and the probability of ending with letter l are:

$$P(x_1 = k) = a_{begin,k}$$

$$P(end | x_n = l) = a_{l,end}$$

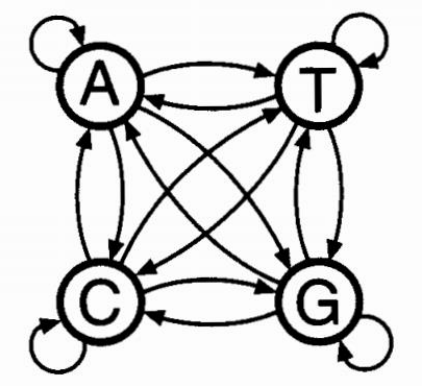


Fig. A. State transition diagram for a Markov model of DNA

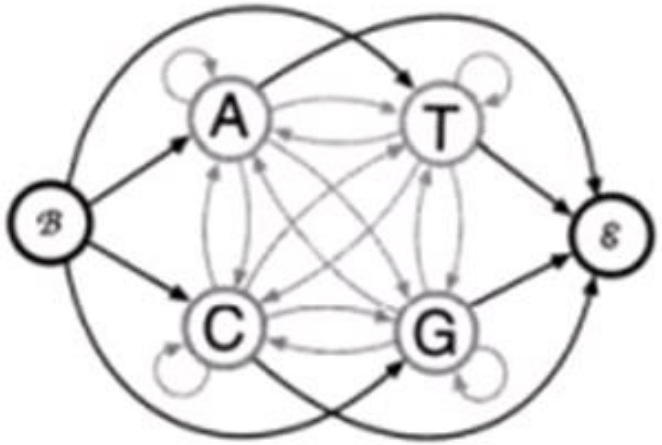


Fig. B. State transition diagram with begin and end states for a Markov model of DNA

Modeling CpG islands (1)

- **Biological background**: When dinucleotide CG (denoted as CpG pair) occurs in a DNA sequence, C is usually modified by methylation and becomes T. As such, CpG pairs are rare except around regulatory regions (e.g. promoters). The regions that are enriched with CpG pairs are called CpG islands
- **Objective**: Use Markov chains to discriminate CpG islands from other DNA regions
- **Data**: From a set of human DNA sequences, a total of 48 putative CpG islands were extracted
- **Modeling**: Two Markov chain models were derived:
  - from the regions labeled as CpG islands (the '+' model), and
  - from the remainder of the sequence (the '–' model)
- The transition probabilities for each model were set using the ML (maximum likelihood) estimates:

$$a_{k,l}^{+} = \frac{n_{k,l}^{+}}{\sum_{l' \in \Omega} n_{k,l'}^{+}}$$

where $n_{k,l}^{+}$ is the number of times letter k is followed by letter l. Likewise $a_{k,l}^{-}$ was found for the '–' model

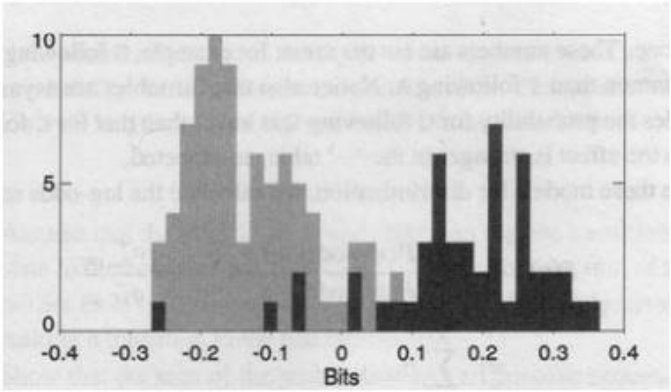
Modeling CpG islands (2)

- To discriminate between CpG islands and other regions, we calculate the log-odds ratio score:

$$S(x) = \log \frac{P(x|\text{model}+)}{P(x|\text{model}-)} = \sum_{i=1}^{n+1} \log \frac{a^{+}_{x_{i-1},x_i}}{a^{-}_{x_{i-1},x_i}} = \sum_{i=1}^{n+1} \beta_{x_{i-1},x_i}$$

- where x is the DNA sequence and $\beta_{x_{i-1}x_i}$ are the log likelihood ratios of corresponding transition probabilities
- A table for β is given below in bits (using base 2 logarithm)
- The distribution of the length-normalized scores $S(x)$ for CpG islands (dark grey) and non-CpG (light grey) is shown in the figure below

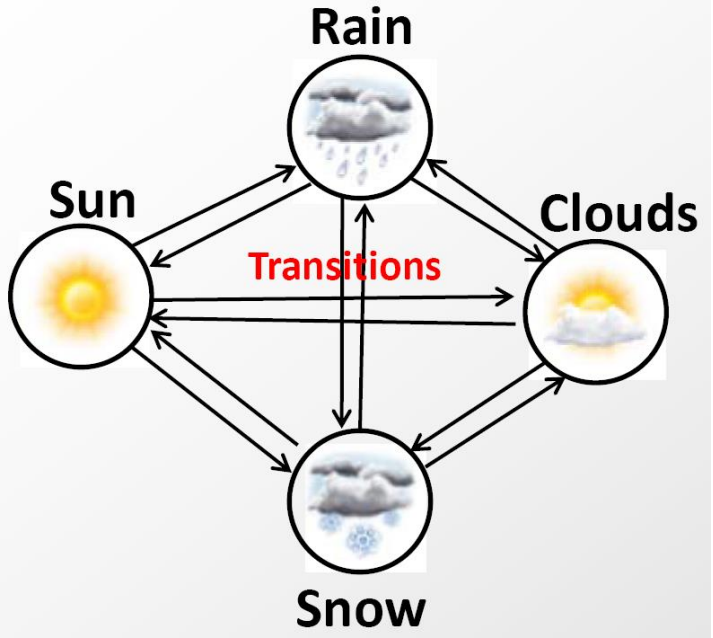
β	A	C	G	T
A	-0.74	0.42	0.58	-0.80
C	-0.91	0.30	1.81	-0.68
G	-0.62	0.46	0.33	-0.72
T	-1.17	0.57	0.39	-0.68



Hidden Markov Models

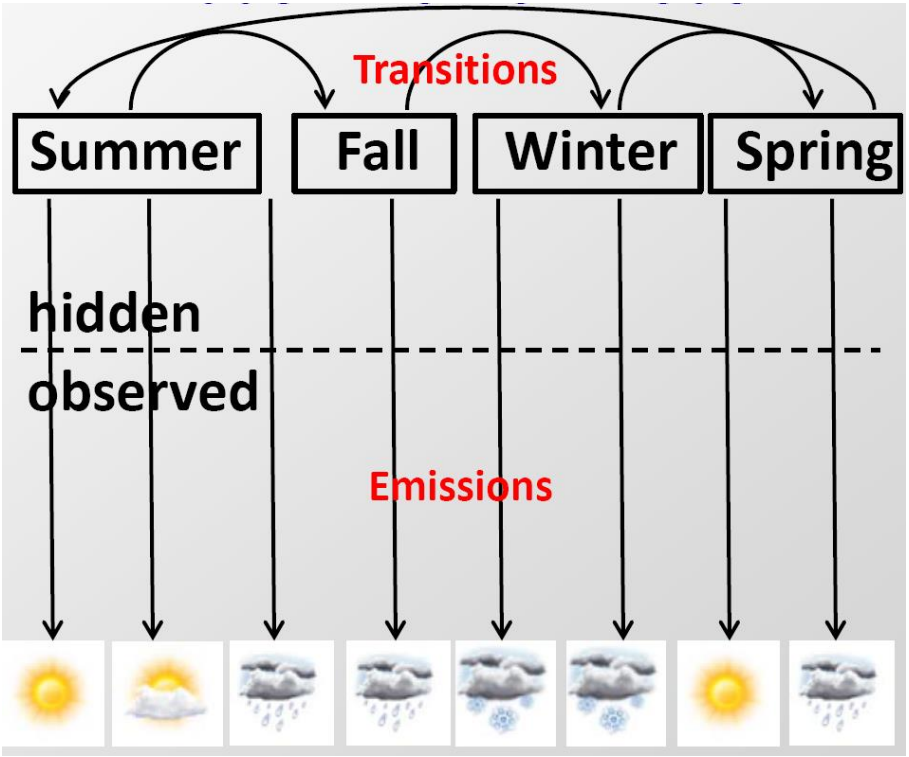
Predicting tomorrow’s weather

Markov Chain



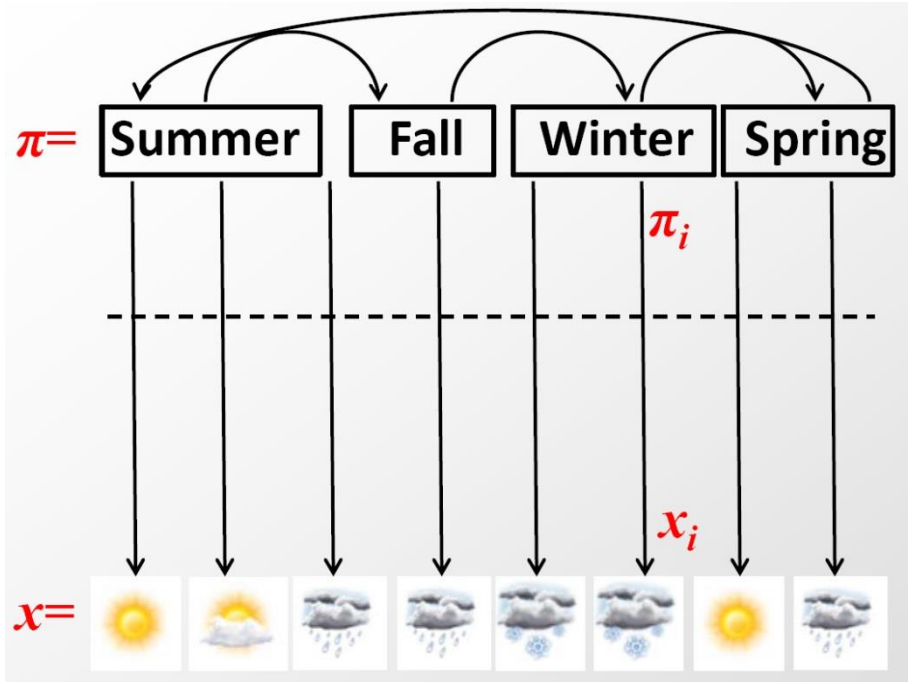
- What you see is what you get: next state only depends on the current state (no memory)

Hidden Markov Model



- Hidden states of the world (e.g. seasons) determine emission probabilities
- State transitions are governed by a Markov chain

Nomenclature



Transitions: $a_{kl} = P(\pi_i = l | \pi_{i-1} = k)$

Transition probability from state k to state l

Emissions: $e_k(x_i) = P(x_i | \pi_i = k)$

Emission probability of generating symbol x_i from state k

- Vector x = sequence of observations
- Vector π = hidden path of states (sequence of hidden states)
- Transition matrix **A** : a_{kl} = probability of $k \rightarrow l$ state transition
- Emission vector **E** : $e_k(x_i)$ = probability of observing x_i from state k

Components of an HMM (hidden Markov model)

- Definition: An **HMM** is a 5-tuple $(\boldsymbol{Q}, \boldsymbol{V}, \boldsymbol{p}, \boldsymbol{A}, \boldsymbol{E})$, where
  - $\boldsymbol{Q}$ is a finite set of states
  - $\boldsymbol{V}$ is a finite set of observed symbols per state
  - $\boldsymbol{p}$ is the initial state probabilities
  - $\boldsymbol{A}$ is the state transition probabilities, denoted by a_{st} for states s, t in Q :

$$a_{st} = P(\pi_i = t | \pi_{i-1} = s)$$

  - $\boldsymbol{E}$ is a matrix of emission probabilities: $e_k(b) = P(x_i = b | \pi_i = k)$

- **Output**: Only emitted symbols are observable by the system, but not the underlying random walk between states, because the states are "hidden"

- **Markov property**: Emissions and transitions are dependent on the current state only and not on the past

Viterbi algorithm (1)

- The joint probability of an observed sequence x and a state sequence π is:

$$P(x,\pi) = a_{0\pi_1}\prod_{i=1}^{L} e_{\pi_i}(x_i)a_{\pi_i\pi_{i+1}}$$

- In applications, we often want to find an optimal state path (i.e. a path with the highest probability):

$$\pi^* = \underset{\pi}{\operatorname{argmax}}\, P(x,\pi)$$

- The optimal path π^* can be found recursively, using a **dynamic programming** algorithm called **Viterbi algorithm**:
  - Suppose the probability $v_k(i)$ of the most probable (i.e. optimal) path ending in state k with observation i is known for all the states k
  - Then, these probabilities can be calculated **recursively** for observation x_{i+1} as

$$v_l(i+1) = e_l(x_{i+1})\max_k(v_k(i)a_{kl})$$

  - $v_k(i)$ is called the **Viterbi variable**

Viterbi algorithm (2)

Algorithm: Viterbi

Initialization ($i = 0$): $v_0(0) = 1, v_k(0) = 0$ for $k > 0$.

Recursion ($i = 1 \ldots L$): $v_l(i) = e_l(x_i) \max_k (v_k(i-1)a_{kl})$;

$\text{ptr}_i(l) = \text{argmax}_k (v_k(i-1)a_{kl})$.

Termination: $P(x, \pi^*) = \max_k (v_k(L)a_{k0})$;

$\pi_L^* = \text{argmax}_k (v_k(L)a_{k0})$.

Traceback ($i = L \ldots 1$): $\pi^*_{i-1} = \text{ptr}_i(\pi_i^*)$.

- Note:
  - All sequences have to start in state 0, so the initial condition is $v_0(0) = 1$
  - An end state is assumed, with also index 0, so there is a_{k0} in the termination step
  - By keeping pointers backwards, the path can be found by backtracking

Forward algorithm (1)

- From the joint probability of sequence and state path

$$P(x,\pi) = a_{0\pi_1}\prod_{i=1}^{L} e_{\pi_i}(x_i)a_{\pi_i\pi_{i+1}}$$

  we would like to calculate the full probability of sequence x for all possible paths

$$P(x) = \sum_{\pi} P(x,\pi)$$

- However, the number of possible paths increases exponentially with the length of the sequence
- One approach is to use the most probably path obtained by Viterbi algorithm, as an approximation of $P(x)$, which often works well. But it is not exactly correct
- We can again use a **dynamic programming** method, similar to Viterbi algorithm, only replacing the maximization step with summation. This is called the **forward algorithm**

Forward algorithm (2)

- The Viterbi variable $v_k(i)$ is replaced by the **forward variable**:

$$f_k(i) = P(x_1 \ldots x_i, \pi_i = k)$$

- The recursion equation is:

$$f_l(i+1) = e_l(x_{i+1}) \sum_k f_k(i) a_{kl}$$

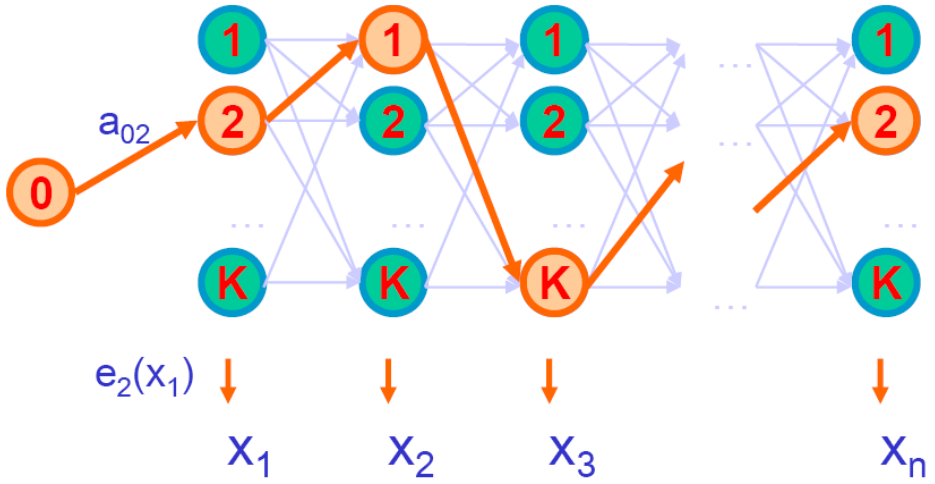
- **Algorithm: Forward algorithm**

Initialization ($i = 0$): $f_0(0) = 1$, $f_k(0) = 0$ for $k > 0$.

Recursion ($i = 1 \ldots L$): $f_l(i) = e_l(x_i) \sum_k f_k(i-1) a_{kl}$.

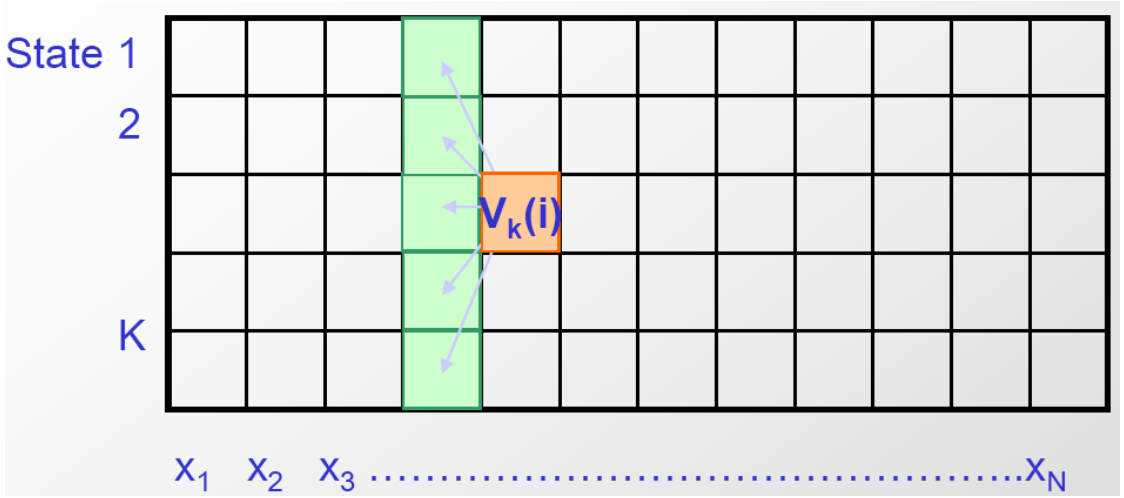
Termination: $P(x) = \sum_k f_k(L) a_{k0}$.

Dynamic programming in HMM

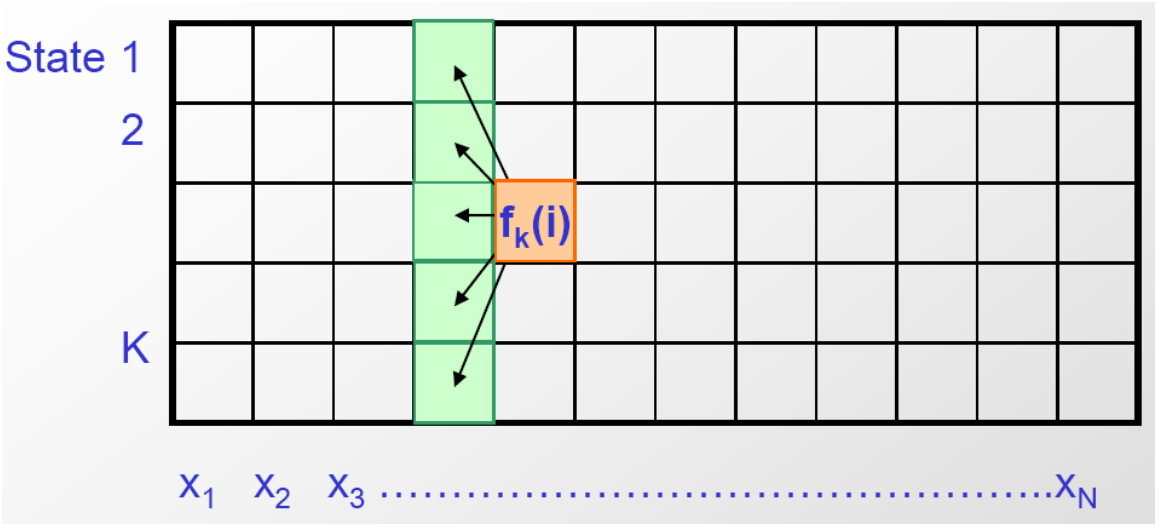


HMMs are generative models:
A sequence can be generated from an HMM by simulating the successive choices of state and emission of a symbol from the state

The Viterbi Algorithm



The Forward Algorithm



Other algorithms of HMM

- To calculate the posterior probability that the observation x_i came from state k given the observed sequence, $P(\pi_i = k|x)$, we want to calculate the joint probability of the sequence with the i-th symbol being produced by state k:

$$P(x, \pi_i = k) = P(x_1 \dots x_i, \pi_i = k)P(x_{i+1} \dots x_L | x_1 \dots x_i, \pi_i = k)$$
$$= P(x_1 \dots x_i, \pi_i = k)P(x_{i+1} \dots x_L | \pi_i = k)$$

  - The first term is the **forward variable** $f_k(i)$
  - The second term is called **backward variable** $b_k(i)$
- Similar to the forward algorithm, the **backward algorithm** is a dynamic programming algorithm for the calculation of $b_k(i)$
- To estimate the parameters of an HMM, i.e. transition and emission probabilities, from a training dataset:
  - When the paths (state sequences) are known for the training data, we can use the maximum likelihood estimator
  - When the paths are unknown for the training data, we can use **Baum-Welch algorithm** (a special case of Expectation Maximization algorithm)

Expectation Maximization

Incomplete data

- When values of all attributes are known (i.e. complete data), learning is relatively easy
- But many real-world problems have **hidden variables** (or **latent factors**), due to:
  - Incomplete data (or missing values)
  - Unknown attributes / relations
- Naive solutions to the issue of missing data
  - Ignore records of missing values: But a hidden variable might have no value at all
  - Ignore hidden variables: But a model may become much more complex

Expectation Maximization (EM)

- **Expectation Maximization (EM)** is an algorithm for ML estimation with missing data. With observed data *x*, missing data *y*, the aim is to find the model that maximizes the log likelihood

$$\log P(x|\theta) = \log \sum_y P(x, y|\theta)$$

- The EM algorithm reduces the optimization of $\log P(x|\theta)$ into a sequence of iterations
- In each iteration, the EM algorithm does :

(1) **E-step**: Calculate $Q(\theta|\theta^t) = \sum_y P(y|x, \theta^t) \log P(x, y|\theta)$

(2) **M-step**: Set θ^{t+1} to value of θ that maximizes $Q(\theta|\theta^t)$

(The next iteration sets $\theta^t \leftarrow \theta^{t+1}$ and repeats)

- It can be proven that the likelihood increases monotonically over the iterations of EM algorithm, converging to a local (or maybe global) maximum as $t \rightarrow \infty$

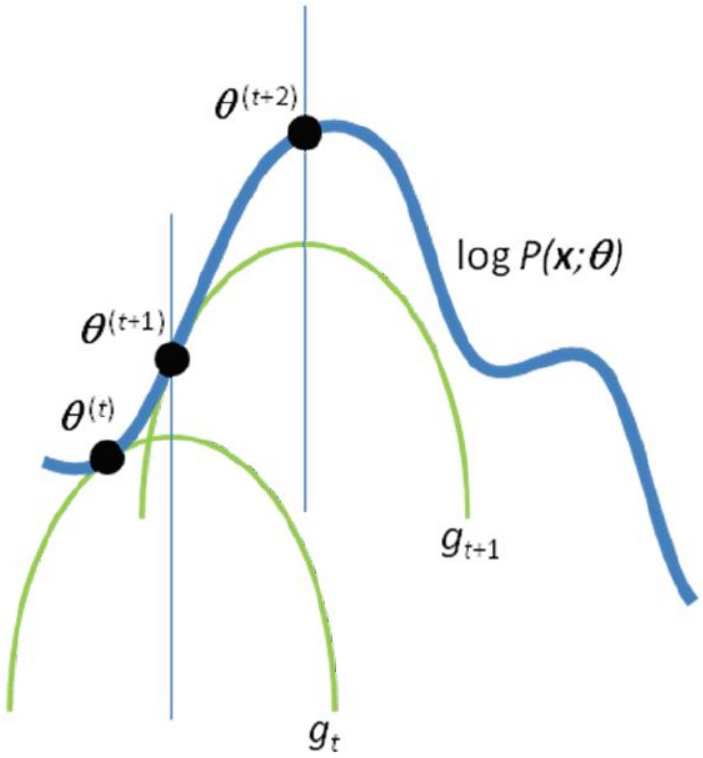


Figure. The convergence of the EM algorithm. Here, g_t function takes the role of Q function

Figure from Do & Batzglou, *Nature Biotech*, 2008

Baum-Welch algorithm for parameter estimation of HMM

- This estimates the parameters when the paths of training sequences are unknown. An iterative procedure is to start from arbitrary values and then the values of the parameters are iteratively improved
- From given sequences, find the probable paths of the training sequences by using the current estimates (and computing the expectations) of $a_{k,l}$ and $b_k(\sigma)$. Then, estimate a's and b's by maximizing the likelihood. This process continues until some stopping criterion is reached
- Baum-Welch is an Expectation Maximization (EM) algorithm

Pseudocode of Baum-Welch algorithm

Baum–Welch Algorithm

Initialization: arbitrary parameters for $a_{k,l}$ and $b_k(\sigma)$

Recurrence:

- Set all counts A (expectation of $a_{k,l}$) and B (expectation of $b_k(\sigma)$) to their pseudo-count values (or to zero)
- For each training sequence $j = 1 \ldots m$: (**E-step**)
  - Calculate $\alpha^j_k(i)$ for position i in sequence j, using the forward algorithm
  - Calculate $\beta^j_k(i)$ for position i in sequence j, using the backward algorithm
  - Add the contribution of sequence j to A and B
- Calculate the new model parameters (**M-step**)
- Calculate the new log-likelihood using the model

Termination:

- Stop if the log-likelihood of the model is greater than some predefined threshold or the maximum number of iteration is reached

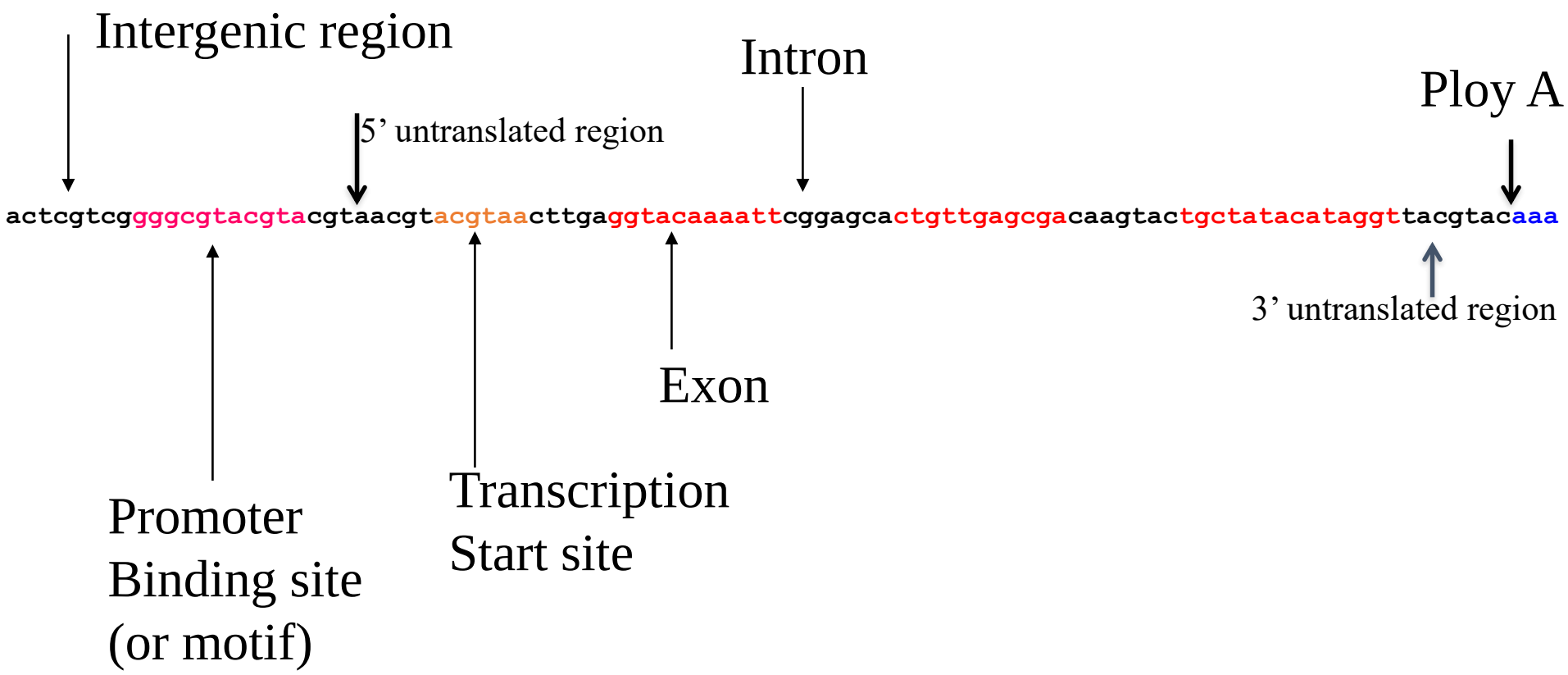
Applications of HMMs in Bioinformatics

Application of HMM in biological sequence analysis

- Gene prediction
- Protein sequence modeling (learning, profile)
- Protein sequence alignment (decoding)
- Protein database search (scoring, e.g. fold recognition)
- Protein structure prediction
- …

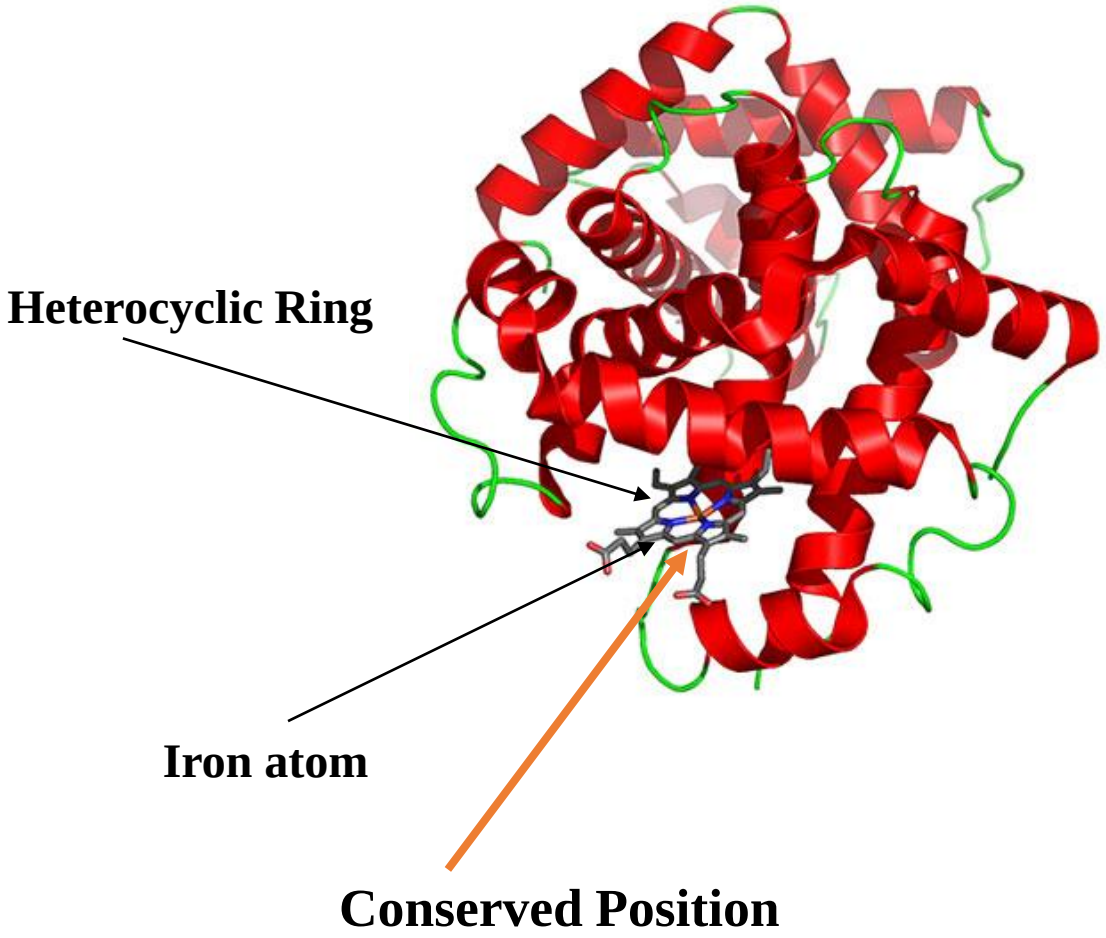
Motif and gene structure

- HMM has been used for modeling binding site and gene structure prediction (e.g. GENESCAN, http://hollywood.mit.edu/GENSCAN.html)



Model protein family (profile HMM)

- Create a statistical model (HMM) for a group of related protein sequences (e.g. protein family)
- Identify core (conserved) elements of homologous sequences
- Positional evolutionary information (e.g. insertion and deletion)



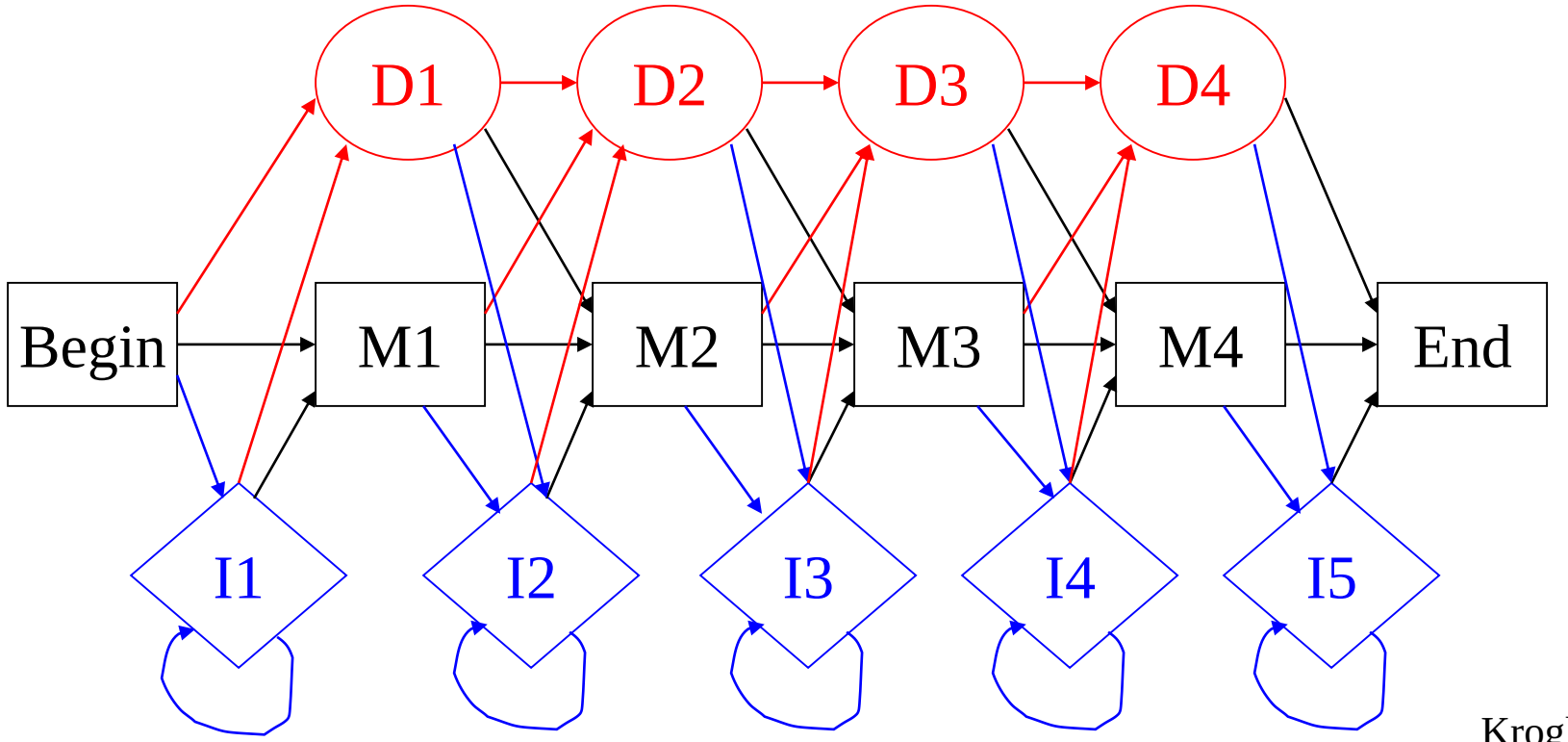
Why do we build a profile?

- Understand the conservation (core function and structure elements) and variation
- Sequence generation
- Multiple sequence alignments
- Profile-sequence alignment (more sensitive than sequence-sequence alignment)
- Family / fold recognition
- Profile-profile alignment

Key to build an HMM is to set up states

- Think about the positions of the ancestral sequence are undergoing mutation events to generate new sequences in difference species:
  - A position can be modeled by a **die**
  - **Match** (match or mutate): the position is kept without or with variations / mutations
  - **Delete**: the position is deleted
  - **Insert**: amino acids are inserted between two positions

Architecture of HMM for protein sequences



Krogh et al. 1994, Baldi et al. 1994

- Each match state has an emission distribution of 20 amino acids
- Each insertion state has an emission distribution of 20 amino acids
- Variants of architecture exist

Estimate parameters (learning)

- We want to find a set of parameters to maximize the probability of the observed sequences in the family:

maximum likelihood P(sequences | model) = P(sequence 1| model) * … * P(sequence n | model)

- Using Baum-Welch’ s algorithm (or EM algorithm)

HMM software and code

- HMMER: http://hmmer.org/
- SAM: http://www.cse.ucsc.edu/research/combio/sam.html
- HHSearch: http://toolkit.tuebingen.mpg.de
- MUSCLE: http://www.drive5.com/muscle/
- HHSuite: https://github.com/soedinglab/hh-suite
- HHblits

Remmert M, Biegert A, Hauser A, Söding J (2011). "HHblits: Lightning-fast iterative protein sequence searching by HMM-HMM alignment.". *Nature Methods*, 9 (2): 173–175

Recap

- In this lecture, we have learned:
  - Basics of Markov chain
  - Basics Hidden Markov model (HMM)
  - Expectation maximization (EM) algorithm
  - How HMMs are applied to bioinformatics

References

- R. Durbin, S. Eddy, A. Krogh, G. Mitchison. “Biological sequence analysis: Probabilistic models of proteins and nucleic acids”, Chapter 3 “Markov chains and hidden Markov models”, Cambridge University Press, 1998.
- P. Baldi, S. Brunak. “Bioinformatics: The machine learning approach”, MIT Press, 2001.
- C. B. Do, S. Batzoglou. “What is the expectation maximization algorithm?”, *Nature Biotechnology*, Vol. 26, No. 8, p. 897-9, 2008.